```
$ cd ..

$ cd test

$ ruby --version
ruby 3.2.0 (2022-12-25 revision a528908271) [arm64-darwin22]
```

Rbenv accomplishes this by putting a file in the directory called .ruby-version, which just contains the version number of the Ruby you've set for that directory.

```
% cat .ruby-version
3.2.0
```

This file also works in reverse—if you have rbenv installed and you change to a directory that contains a .ruby-version file, rbenv will automatically change to that versions Ruby if installed, or warn you that the directory expects an uninstalled Ruby if it is not. Many Ruby projects use a .ruby-version file to specify their Ruby version, and it is respected by all the Ruby version managers.

If you want to set a default Ruby version for directories that don't specify their own, you can do so with rbenv global <version>.

This may be more work than you were expecting to install Ruby. If all you ever wanted to do was use a single version of Ruby, we'd agree. But what you've really done here is given yourself an incredible amount of flexibility. Maybe in the future a project comes along that uses Ruby 2.7.5, per its .ruby-version file. That's not a problem—just use rbenv install 2.7.5 and rbenv rehash and rbenv will automatically pick up the version from the .ruby-version file.

What Is rbenv Actually Doing?

Rbenv attempts to provide its dynamic behavior with as little change to your regular terminal environment as possible.

A Unix terminal uses a global environment variable called PATH to determine what directories it looks in for executable programs when you type a command. If you look in your configuration file for your terminal, you will likely see the PATH variable being modified.

When the rbenv init command is executed as part of your terminal setup, it inserts a directory at the front of your PATH, so that your operating system will look in the rbenv directory before looking anyplace else. In that directory are a set of what rbenv calls *shims*; small programs that match all the executable commands in all your Ruby versions. (The reason why you need to run rbenv rehash after installing a new Ruby is to refresh this directory). When you call a Ruby command, like ruby or (as you'll see in a a minute) irb, the rbenv shim is encountered first, and dynamically chooses which Ruby is active, usually based on the presence of a .ruby-version file. Then the command is handed off to the actual executable program in that current version. You can see those actual versions, they live in your home directory at ~/.rbenv/versions.

Installing Ruby For Windows

Ruby is not available as a default option in Windows the way it is in Unix distributions or MacOS, but it can be installed and used and can interact with the underlying environment to automate Windows-specific resources.

We're going to focus on two ways to install Ruby on Windows: using the Windows Subsystem for Linux (WSL),[10] which allows you to run a Linux command-line terminal in your Windows

10. https://docs.microsoft.com/en-us/windows/wsl